Adventist University of Central Africa
P.O. Box 2461 Kigali, Rwanda | www.auca.ac.rw | info@auca.ac.rw

Department of Information Technology

Course Code and Name: INSY 8311 – Database Development with PL/SQL

Assignment 3: Cursors and Advanced Data Processing (Group Assignment)

Released Date: Sunday, 5th October 2026

Deadline: Sunday, 18th October 2026, 11:59 PM

1. Purpose

In Assignment 2 you wrote blocks that read one row at a time with SELECT INTO. Real programs usually need to process many rows, and that is what cursors are for. This assignment covers implicit cursors and their attributes, explicit cursors (DECLARE, OPEN, FETCH, CLOSE), %ROWTYPE records, cursor FOR loops, cursors with parameters, cursor variables (REF CURSOR / SYS_REFCURSOR) and FOR UPDATE with WHERE CURRENT OF. It uses the same schema you already set up in your XEPDB1 database as the **lecturer** user: **STUDENTS**, **COURSES**, **ENROLLMENTS**, **EMPLOYEES**, and **DEPARTMENTS**.

Section A is a recap of the previous chapters: one SQL question and one PL/SQL question on variables, conditions and loops. Every cursor is built on a SELECT statement, and every cursor loop uses the IF, CASE and loop skills from Assignment 2, so these concepts must stay sharp. All other tasks must be solved with PL/SQL anonymous blocks that print their results with DBMS_OUTPUT.PUT_LINE.

2. Submission Rules: Read Before You Start (Git Workflow)

Follow this exactly. Any deviation results in disqualification of your submission.

1. **Create your own branch** dedicated to this assignment. Use the naming convention: **student_id1_student_id2_assignment3** (e.g. **20657_20658_assignment3**)
2. Do all your work on **that branch only**.
3. **Commit** your work with clear commit messages.
4. **Push your branch** to the remote repository.
5. **Do NOT push to main** and do NOT open a pull request that merges into main yourself. Submission is your pushed branch, nothing else.
6. Submitting via your own repository, a zip file, email, or any channel other than a pushed branch on the class repo will **not be graded**.

Disqualifying actions: pushing to main, creating a personal/separate repository for this work, or submitting outside your dedicated branch.

What to push

A single, dedicated folder for this assignment inside your branch:

1. Assignment3 folder with a .sql file, a README.md and a pdf report. Both the .sql file and the pdf report should follow the same naming convention as your branch.

Each block in the .sql file must start with a comment header like the one below, followed by comments explaining the code you have used:

```
-- ========================================
-- TASK C1: Explicit Cursor with OPEN-FETCH-CLOSE
-- Student: [Your Name] | ID: [Your ID]
-- ========================================
```

The **pdf report** must contain every query and block you wrote together with a screenshot of its output, the approach you used, the lessons you learned and the challenges you faced. The **README.md** must contain the steps someone might use to right your .sql

The README.md and pdf report must be written by you. They must reflect your own testing process for your own code. Copying another student's report, or generating it with an AI tool instead of describing your own verification steps, will be treated as an academic integrity violation.

3. Ground Rules

- ✓ Every query and block must compile and run without error against the schema from the install/config guide (50 students, 15 courses, 75 enrollments, 40 employees, 6 departments). Run SET SERVEROUTPUT ON before testing.
- ✓ Use the cursor technique the task asks for. Where a task says "explicit cursor with OPEN, FETCH and CLOSE", a cursor FOR loop is not accepted, and the other way round. Solving a task a different way earns partial credit at best.
- ✓ Every explicit cursor you OPEN must be CLOSED, including when an error occurs (close it in the EXCEPTION section if it is still open). Every FETCH loop must have an EXIT WHEN cursor_name%NOTFOUND.
- ✓ Declare the variables that receive fetched rows with %TYPE or %ROWTYPE. Never use SELECT INTO for a query that can return more than one row.
- ✓ Tasks that change data (B1, B2 and F1) must end with ROLLBACK, so that every student's database stays identical to the lab setup for the other tasks. Say in a comment what COMMIT would have done instead.
- ✓ Do not hardcode values you could otherwise derive (e.g. don't type a course average; calculate it). The IDs and names given in a task statement may be used as given.
- ✓ Code that does not compile receives 0 for that task. Comment your code: a correct block with zero explanation of intent loses style points.

4. Tasks (100 points)

Section A: Recap of Previous Chapters (15 pts)

A1 (5 pts). SQL. Write one plain SQL query (no PL/SQL) that lists every student who has at least one graded enrollment (score IS NOT NULL), showing full name, number of graded courses, average score (rounded to 1 decimal place) and highest score. Keep only students whose average score is 85 or above, add a column that ranks them by average score using RANK(), and order the result by that rank. In your README, state how many students your query returned.

A2 (10 pts). PL/SQL Fundamentals (no cursors). Using a numeric FOR loop over course_id 101 to 115, SELECT INTO and variables declared with %TYPE (this is the Chapter 2 approach, so explicit cursors and cursor FOR loops are not allowed here), print one line per course showing the course code, credits, number of enrolled students and a fill rate (enrolled / max_students * 100, rounded to 1 decimal place). Use a CASE to label each course by fill rate:

- below 20%: 'Low Demand'
- 20% to below 50%: 'Moderate Demand'
- 50% or more: 'High Demand'

Use IF statements to keep a count for each label and print the three counts after the loop. In your README, explain in two or three sentences why this approach runs one query per course, and which cursor from this assignment would do the same work more efficiently.

Section B: Implicit Cursors (10 pts)

B1 (5 pts). SQL%ROWCOUNT and SQL%FOUND. Write a block that sets the status of every graded enrollment in the Mathematics department's courses to 'Completed' with a single UPDATE (find the courses with a subquery on COURSES.department; do not type the course IDs). Immediately after the UPDATE, use SQL%FOUND and SQL%ROWCOUNT to print either 'X enrollment(s) marked as Completed' or 'No enrollments updated'. End with ROLLBACK and print 'Changes rolled back'.

B2 (5 pts). SQL%NOTFOUND. Declare two variables, v_student_id and v_course_id, and write a DELETE that removes the enrollment matching both. Use SQL%NOTFOUND to print 'No enrollment found for student X in course Y' when nothing was deleted, and 'Enrollment removed' otherwise. Run the block twice: once with student 1050 and course 999, and once with student 1050 and course 105. End each run with ROLLBACK. In a comment, explain why SQL%ISOPEN always returns FALSE.

Section C: Explicit Cursors and Cursor Attributes (20 pts)

C1 (10 pts). OPEN, FETCH, CLOSE with %ROWTYPE. Declare an explicit cursor that returns the student_id, full name, email and GPA of every student in the 2024-09-01 cohort (STUDENTS.enrollment_date = DATE '2024-09-01'), ordered by GPA from highest to lowest.

Declare one record variable with cursor_name%ROWTYPE. Using OPEN, a basic LOOP with FETCH and EXIT WHEN %NOTFOUND, and CLOSE, print a numbered list like the one below, then the total number of students and the cohort's average GPA (calculated inside the loop, not with AVG).

```
1. [Full Name] ([student_id]) | [email] | GPA [x.x]
2. ...
```

C2 (10 pts). Cursor Attributes. Declare an explicit cursor over EMPLOYEES ordered by salary, highest first. Before opening it, use %ISOPEN to print the cursor state, then open it and print the state again. Fetch rows until you have printed the top 5 earners, using %ROWCOUNT (not a separate counter variable) both to number each line and to stop the loop. After the loop, print the value of %ROWCOUNT, close the cursor and print the state a third time. In a comment, explain why %ROWCOUNT after the loop is not the number of employees in the table.

Section D: Cursor FOR Loops and Cursors with Parameters (20 pts)

D1 (10 pts). Course Roster. Declare a cursor with one parameter, p_course_id, that returns the full name, score and grade of every student enrolled in that course, ordered by score (highest first, ungraded last). Use a cursor FOR loop to print a roster for each of the courses 101, 103 and 110. For each course print a heading with the course name, one line per student (show 'Not graded' for a NULL score), then the number of students, the number graded and the average of the graded scores. If a course has no students, print 'No students enrolled'.

D2 (10 pts). Staff Above Their Department Average. Use an outer cursor FOR loop over DEPARTMENTS. For each department, calculate its average salary, then open a second cursor with two parameters, p_department and p_min_salary, that returns the employees of that department earning more than p_min_salary. Pass the department's average as p_min_salary and print those employees with their salary and how far above the average they are. Departments with no staff must print 'No staff in this department'. At the end, print the total number of employees listed across all departments.

Section E: Cursor Variables (10 pts)

E1 (10 pts). One Cursor, Three Reports. Declare a variable of type SYS_REFCURSOR and a variable v_report that holds one of 'HONOURS', 'PROBATION' or 'NEW_INTAKE'. Use IF/ELSIF (or CASE) to OPEN the cursor variable FOR a different query in each case, each returning the same columns (student_id, full name, GPA, enrollment_date):

- 'HONOURS': students with a GPA of 3.7 or above, highest GPA first;
- 'PROBATION': students with a GPA below 3.0, lowest GPA first;
- 'NEW_INTAKE': students whose enrollment_date is on or after 1 September 2024, most recent first;
- any other value: print 'Unknown report type' without opening the cursor.

Fetch and print the rows with one common loop, then print the row count using the cursor variable's %ROWCOUNT and close it. Run your block with each of the four cases and include all four outputs in your pdf report.

Section F: FOR UPDATE and WHERE CURRENT OF (10 pts)

F1 (10 pts). Score Curve for Introduction to Programming. The instructor of course 101 has approved a curve: every graded score below 80 gets 3 extra points (never above 100), and its letter grade is recalculated with the scale from Assignment 2 (90 and above A, 80 to 89.99 B, 70 to 79.99 C, 60 to 69.99 D, below 60 F). Declare a cursor over the graded enrollments of course 101 with a score below 80, using FOR UPDATE OF score, grade. In a cursor FOR loop, apply the curve with UPDATE ... WHERE CURRENT OF, and print each change as shown below. After the loop print the number of enrollments curved, then ROLLBACK. In a comment, explain what FOR UPDATE protects against and when the row locks are released.

```
Student 1040: 72.5 (C) -> 75.5 (C)
Student 1008: 78.5 (B) -> 81.5 (B)
```

Section G: Combined Challenge (15 pts)

G1 (15 pts). Computer Science Course Performance Report. Write a single anonymous block that produces a performance report for every course offered by the Computer Science department. The block must use:

- **an explicit outer cursor** (OPEN, FETCH, CLOSE) over the Computer Science courses, in course_code order;
- **an inner cursor with a parameter** (p_course_id), processed with a cursor FOR loop, over that course's enrollments;
- **a CASE** that labels each course 'Strong' (average score 85 or above), 'Satisfactory' (75 to 84.99) or 'Needs Attention' (below 75), and 'No results yet' if nothing is graded.

For each course, print the course code and name, instructor, enrolled students versus max_students, the number graded, the minimum, maximum and average score, the label, and the names of the students scoring below the course average. After the last course, print a summary with the number of courses processed, the course with the highest average score and the total number of below-average students found. Compute all figures inside the loops; do not use a separate aggregate query.

A perfect answer submitted outside the correct branch, or pushed to main, receives **0** for the assignment regardless of code quality. **Late submissions, if no extension was given, lose 10 points per day for three days. If an extension was given, late submission will not be accepted.**

5. Academic Integrity

You may discuss approaches with classmates, but the SQL and PL/SQL you submit must be your own work, written and tested by you against your own database instance. Identical, unexplainable code files across two students will be treated as an academic integrity violation for both parties.

This includes the README.md and pdf report. They must be written in your own words, describing what you actually did and observed. A report that is copied, templated, or produced by an AI tool rather than reflecting your own verification work will be treated the same as copied code: an academic integrity violation.